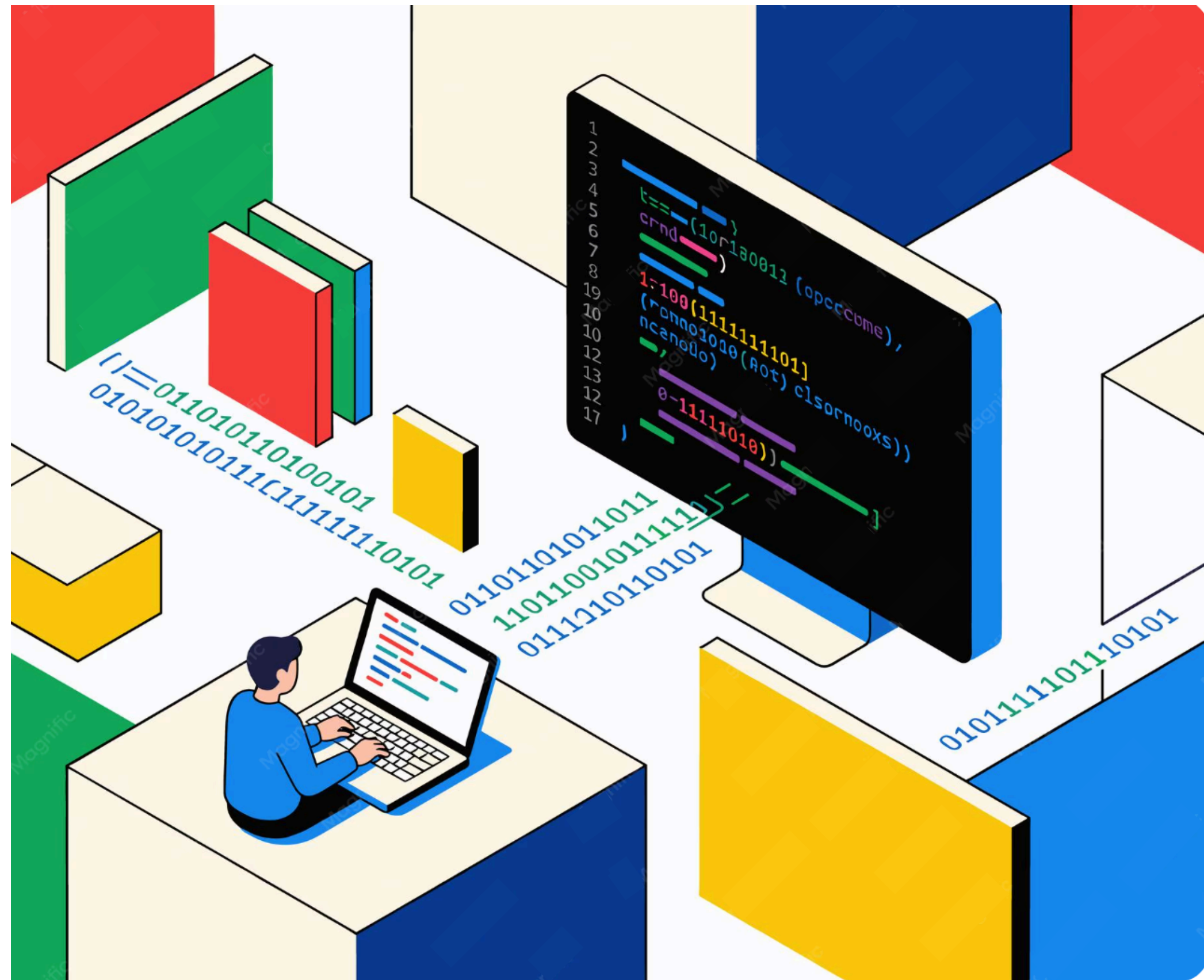


Pemrograman Berbasis Objek

Property Visibility & Class Method

Semester 2026/2027

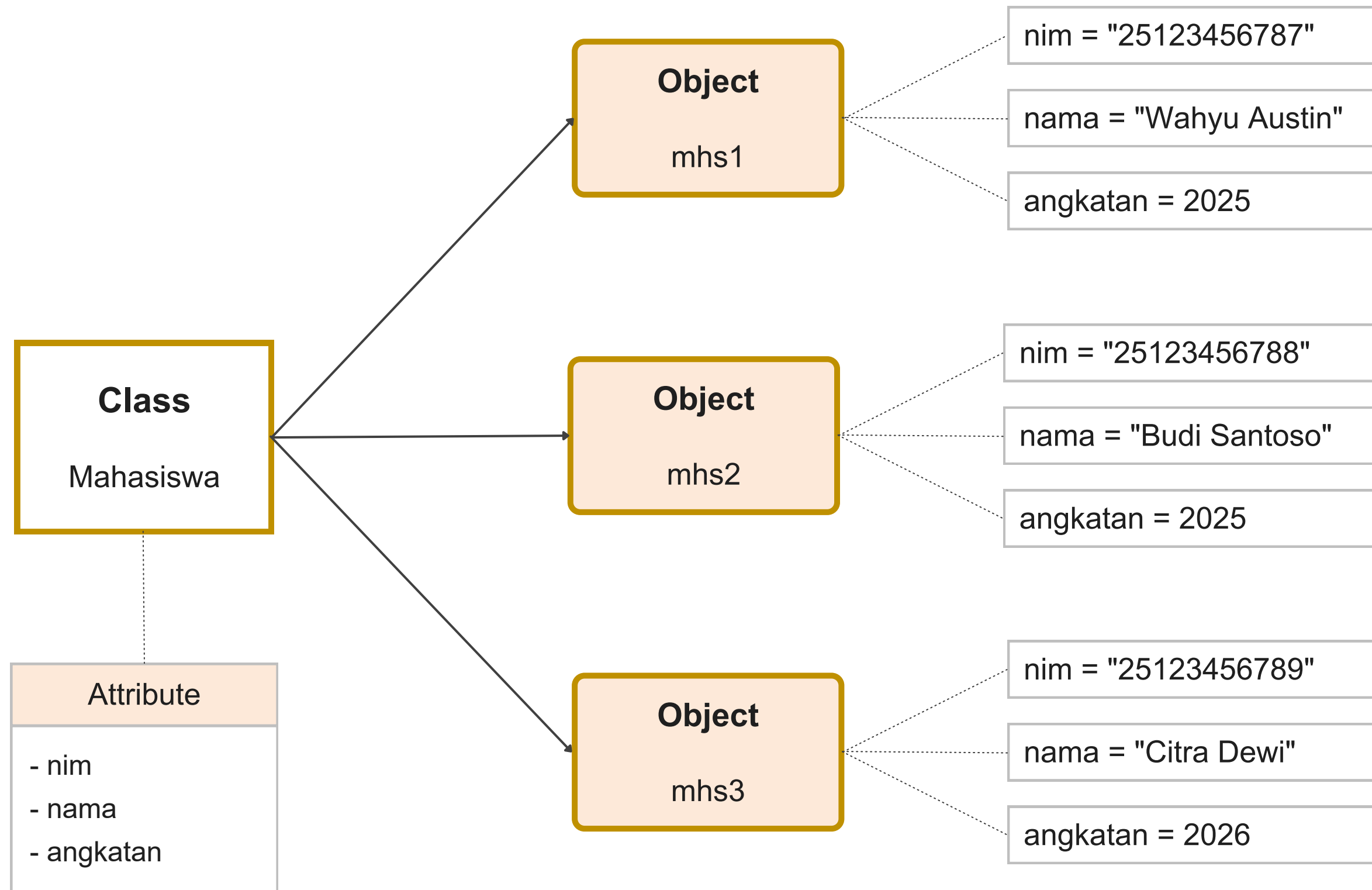
07 September 2026



Let's Recap !

Review materi pada pertemuan sebelumnya

Let's Recap !



Let's Code !

- Buatlah class dan object dengan rincian sebagai berikut:
 - Nama class : “Mobil”
 - Atribut :
 - Nama Merek
 - Perusahaan Produksi
 - Tahun Produksi
 - Instance Method :
 - Tampilkan info
 - Tambah Kilometer
 - Cek Kondisi mesin

Let's Recap !

- Class “Mobil”

```
class Mobil:

    def __init__(self, nama_merek, perusahaan_produksi,
        tahun_produksi):
        self.nama_merek = nama_merek
        self.perusahaan_produksi = perusahaan_produksi
        self.tahun_produksi = tahun_produksi
        self.kondisi_mesin = 100
        self.kilometer = 0

    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Perusahaan : {self.perusahaan_produksi}")
        print(f"Tahun : {self.tahun_produksi}")
        print(f"Kilometer : {self.kilometer} km")
        print(f"Kondisi mesin: {self.kondisi_mesin}%")

    def tambah_kilometer(self, kilometer):
        self.kilometer += kilometer
```

```
    def cek_kondisi_mesin(self):
        if self.kondisi_mesin >= 80:
            return "Mesin dalam kondisi baik."
        elif self.kondisi_mesin >= 50:
            return "Mesin perlu diperiksa."
        else:
            return "Mesin perlu segera diservis."

    def service(self):
        self.kondisi_mesin = 100
        self.kilometer = 0
        print(f"{self.nama_merek} selesai diservis.")

    def cek_service(self):
        if self.kilometer >= 10000:
            return "Mobil perlu melakukan service."
        else:
            return "Mobil belum perlu service."
```

Let's Recap !

- Object “Mobil”

```
mobil1 = Mobil("Civic", "Honda", 2022)
mobil1.tambah_kilometer(12000)
print(mobil1.cek_service())
print(mobil1.cek_kondisi_mesin())
mobil1.service()
mobil1.tampilkan_info()
```

Argument method: positional, optional, keyword arg

- Aturan-aturan dalam **deklarasi parameter** dan **pengisian argument fungsi** juga berlaku pada method, diantaranya:

- Parameter method yang **memiliki default value**:

```
class Mobil:
    # ...
    def set_details(self, year=2002, description=""):
        self.year = year
        self.description = description
    # ...
```

- Positional** argument :

```
mobil1 = Mobil()
mobil1.name = "M3 GTR"
mobil1.manufacturer = "BMW"
mobil1.set_details(2001, "Best car in NFS Most Wanted")
```

- Optional** argument:

```
mobil2 = Mobil()
mobil2.name = "RX-8"
mobil2.manufacturer = "Mazda"
mobil2.set_details(description="Best car in NFS Underground")
```

- Keyword** argument:

```
mobil3 = Mobil()
mobil3.name = "Le Mans Quattro"
mobil3.manufacturer = "Audi"
mobil3.set_details(description="Best car in NFS Carbon",
                    year=2003)
```

Argument method: args & kwargs

- Seperti fungsi, method juga bisa berisi parameter **args** maupun **kwargs**.

```
class Mobil:
    # ...
    def set_details(self, **param):
        for key in param:
            if key == "name":
                self.name = param[key]
            if key == "manufacturer":
                self.manufacturer = param[key]
            if key == "year":
                self.year = param[key]
            if key == "description":
                self.description = param[key]
    # ...
```

- Melalui parameter ****param**, kita dapat menentukan attribute mana yang akan diisi nilainya secara dinamis.

```
mobil4 = Mobil()
mobil4.set_details(name="Chiron Sport",
                  manufacturer="Bugatti")
mobil4.set_details(year=2021)
mobil4.set_details(description="Best car in NFS Unbound")
mobil4.info()
```

Property Visibility

- **Visibility** atau privacy dalam konteks OOP merujuk pada penentuan apakah **property** (baik itu attribute atau method) dapat **diakses** secara **public** atau hanya bisa diakses dari **dalam class (private)**.
- Secara default attribute dan method bersifat public.
- Namun Python menggunakan konvensi tertentu, salah satunya **prefix __**, untuk menunjukkan property yang sebaiknya tidak digunakan langsung dari luar class.
- Misalkan `__name`, `__list_items`, dan sejenisnya.

```
class Mobil:

    def __init__(self, nama_merek, perusahaan_produksi,
                  tahun_produksi):
        self.nama_merek = nama_merek
        self.perusahaan_produksi = perusahaan_produksi
        self.__tahun_produksi = tahun_produksi
        self.__kondisi_mesin = 100
        self.__kilometer = 0

mobil1 = Mobil("Civic", "Honda", 2022)
print(mobil1.__tahun_produksi)
```

Property Visibility

- **Visibility** atau privacy dalam konteks OOP merujuk pada penentuan apakah **property** (baik itu attribute atau method) dapat **diakses** secara **public** atau hanya bisa diakses dari **dalam class (private)**.
- Secara default attribute dan method bersifat public.
- Namun Python menggunakan konvensi tertentu, salah satunya **prefix __**, untuk menunjukkan property yang sebaiknya tidak digunakan langsung dari luar class.
- Misalkan `__name`, `__list_items`, dan sejenisnya.

```
class Mobil:

    def __init__(self, nama_merek, perusahaan_produksi,
                  tahun_produksi):
        self.nama_merek = nama_merek
        self.perusahaan_produksi = perusahaan_produksi
        self.__tahun_produksi = tahun_produksi
        self.__kondisi_mesin = 100
        self.__kilometer = 0

mobil1 = Mobil("Civic", "Honda", 2022)
print(mobil1.__tahun_produksi)
```

AttributeError: 'Mobil' object has no attribute '__tahun_produksi'

Property Visibility

- Kita masih bisa menggunakan attribute tersebut dari dalam class, __ digunakan sebagai mekanisme name mangling dan merupakan penanda bahwa property tersebut dimaksudkan untuk penggunaan internal class

```
class Mobil:

    def __init__(self, nama_merek, perusahaan_produksi,
                 tahun_produksi):
        self.nama_merek = nama_merek
        self.perusahaan_produksi = perusahaan_produksi
        self.__tahun_produksi = tahun_produksi
        self.__kondisi_mesin = 100
        self.__kilometer = 0

    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Perusahaan : {self.perusahaan_produksi}")
        print(f"Tahun : {self.__tahun_produksi}")
        print(f"Kilometer : {self.__kilometer} km")
        print(f"Kondisi mesin: {self.__kondisi_mesin}%")

mobil1 = Mobil("Civic", "Honda", 2022)
mobil1.tampilkan_info()
```

Property Visibility

- **Encapsulation** adalah prinsip OOP untuk **membungkus data dan operasi** yang bekerja pada data tersebut dalam satu unit (class), sekaligus **mengendalikan bagaimana data tersebut dapat diakses atau diubah** dari luar object.
- Sehingga perubahan data harus melewati aturan/method yang ditentukan oleh class.

Apa tujuan encapsulation?
→ **Membungkus data dan behavior serta mengontrol akses terhadap data internal.**

```
class Mobil:

    def __init__(self, nama_merek, perusahaan_produksi,
                  tahun_produksi):
        self.nama_merek = nama_merek
        self.perusahaan_produksi = perusahaan_produksi
        self.__tahun_produksi = tahun_produksi
        self.__kondisi_mesin = 100
        self.__kilometer = 0

    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Perusahaan : {self.perusahaan_produksi}")
        print(f"Tahun : {self.__tahun_produksi}")
        print(f"Kilometer : {self.__kilometer} km")
        print(f"Kondisi mesin: {self.__kondisi_mesin}%")

mobil1 = Mobil("Civic", "Honda", 2022)
mobil1.tampilkan_info()
```

Let's Code !

- Buatlah class dan object dengan mengimplementasikan **encapsulation** dengan rincian sebagai berikut:
 - Nama class : "Rekening_Bank"
 - Atribut :
 - Nama Nasabah
 - Nomor Rekening
 - Saldo
 - Instance Method :
 - Tampilkan Info
 - Setor Dana
 - Tarik Dana
 - Cek Saldo

Class Method

- **Class method** adalah method yang **pemilikinya adalah class** dengan pengaksesan adalah via class, berbeda dibanding **instance method** yang diperuntukan untuk **instance object**.
- Jika instance method memiliki parameter self yang isinya adalah instance object, maka class method memiliki **parameter cls** yang isinya adalah tipe data class.
- Jika dilihat dari segi deklarasinya, class method dan instance method memiliki perbedaan berikut:
 - **Instance method** memiliki parameter pertama bernama **self**, dengan isi adalah instance object.
 - **Class method** memiliki parameter pertama bernama **cls** yang merupakan alias dari class dimana method tersebut dideklarasikan (misalnya class method dideklarasikan dalam class Car, maka parameter cls berisi tipe data class Car).
 - Selain perbedaan di atas, class method dideklarasikan dengan menuliskan decorator **@classmethod**

```
class Mobil:
    jumlah_mobil = 0
    semua_mobil = []
    def __init__(self, nama_merek, model, tahun_produksi):
        self.nama_merek = nama_merek
        self.model = model
        self.tahun_produksi = tahun_produksi
        Mobil.jumlah_mobil += 1
        Mobil.semua_mobil.append(self)
```

Instance Method

```
def tampilkan_info(self):
    print(f"Merek : {self.nama_merek}")
    print(f"Model : {self.model}")
    print(f"Tahun : {self.tahun_produksi}")
```

```
def ubah_model(self, model_baru):
    self.model = model_baru
```

Class Method

```
@classmethod
def total_mobil(cls):
    return cls.jumlah_mobil
```

Class Method

```
class Mobil:
    jumlah_mobil = 0
    def __init__(self, nama_merek, model, tahun_produksi):
        self.nama_merek = nama_merek
        self.model = model
        self.tahun_produksi = tahun_produksi
        Mobil.jumlah_mobil += 1

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Model : {self.model}")
        print(f"Tahun : {self.tahun_produksi}")

    def ubah_model(self, model_baru):
        self.model = model_baru

    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil
```

- Panggil **Instance Method**

```
mobil1 = Mobil("Toyota", "Avanza", 2022)
mobil2 = Mobil("Honda", "Civic", 2023)

print(mobil1.tampilkan_info())
print(mobil2.tampilkan_info())
```

- Output :

```
Merek : Toyota
Model : Avanza
Tahun : 2022
Merek : Honda
Model : Civic
Tahun : 2023
```

Class Method

```
class Mobil:
    jumlah_mobil = 0
    def __init__(self, nama_merek, model, tahun_produksi):
        self.nama_merek = nama_merek
        self.model = model
        self.tahun_produksi = tahun_produksi
        Mobil.jumlah_mobil += 1

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Model : {self.model}")
        print(f"Tahun : {self.tahun_produksi}")

    def ubah_model(self, model_baru):
        self.model = model_baru

    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil
```

- Panggil **Instance Method** (ubah_model)

```
mobil1 = Mobil("Toyota", "Avanza", 2022)
mobil2 = Mobil("Honda", "Civic", 2023)

mobil1.ubah_model("Innova")
print(mobil1.tampilkan_info())
print(mobil2.tampilkan_info())
```

- Output :

```
Merek : Toyota
Model : Innova
Tahun : 2022
Merek : Honda
Model : Civic
Tahun : 2023
```

Class Method

```
class Mobil:
    jumlah_mobil = 0
    def __init__(self, nama_merek, model, tahun_produksi):
        self.nama_merek = nama_merek
        self.model = model
        self.tahun_produksi = tahun_produksi
        Mobil.jumlah_mobil += 1

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Model : {self.model}")
        print(f"Tahun : {self.tahun_produksi}")

    def ubah_model(self, model_baru):
        self.model = model_baru

    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil
```

- Panggil **Class Method** (total_model)

```
mobil1 = Mobil("Toyota", "Avanza", 2022)
mobil2 = Mobil("Honda", "Civic", 2023)
mobil3 = Mobil("Toyota", "Innova", 2024)

print(Mobil.total_mobil())
```

- Output :

3

Let's Code !

- Buatlah class dan object dengan mengimplementasikan **class method** dengan rincian sebagai berikut:
 - Nama class : "Mobil"
 - Atribut :
 - Merek mobil
 - Model mobil
 - Tahun
 - Harga sewa per hari
 - Instance Method :
 - Tampilkan Info
 - Hitung biaya sewa
 - Ubah harga sewa
 - Class Method :
 - Total mobil
 - Sewa termurah
 - Sewa termahal

Let's Code !

```
class Mobil:
    jumlah_mobil = 0
    semua_mobil = []
    def __init__(self, merek, model, tahun, sewa_perhari):
        self.merek = merek
        self.model = model
        self.tahun = tahun
        self.__sewa_perhari = sewa_perhari
        Mobil.jumlah_mobil += 1
        Mobil.semua_mobil.append(self)

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.merek}")
        print(f"Model : {self.model}")
        print(f"Tahun : {self.tahun}")

    def hitung_sewa(self, total_hari):
        return total_hari * self.__sewa_perhari
```

```
    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil

    @classmethod
    def sewa_termurah(cls):
        termurah = cls.semua_mobil[0].__sewa_perhari
        for i in range(0, len(cls.semua_mobil)-1):
            if cls.semua_mobil[i+1].__sewa_perhari < termurah:
                termurah = cls.semua_mobil[i+1].__sewa_perhari
        return termurah

mobil1 = Mobil("Toyota", "Avanza", 2022, 100000)
mobil2 = Mobil("Honda", "Civic", 2023, 150000)
mobil3 = Mobil("Toyota", "Mazda", 2024, 200000).

print(f"Rp {mobil1.hitung_sewa(10)}")
print(Mobil.total_mobil())
print(Mobil.sewa_termurah())
```

Let's Code !

```
class Mobil:
    jumlah_mobil = 0
    semua_mobil = []
    def __init__(self, merek, model, tahun, sewa_perhari):
        self.merek = merek
        self.model = model
        self.tahun= tahun
        self.__sewa_perhari = sewa_perhari
        Mobil.jumlah_mobil += 1
        Mobil.semua_mobil.append(self)

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.merek}")
        print(f"Model : {self.model}")
        print(f"Tahun : {self.tahun}")

    def hitung_sewa(self, total_hari):
        return total_hari*self.__sewa_perhari
```

```
    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil

    @classmethod
    def sewa_termurah(cls):
        return min(mobil.__sewa_perhari for mobil in
        cls.semua_mobil)

mobil1 = Mobil("Toyota", "Avanza", 2022, 100000)
mobil2 = Mobil("Honda", "Civic", 2023, 150000)
mobil3 = Mobil("Toyota", "Mazda", 2024, 200000).

print(f"Rp {mobil1.hitung_sewa(10)}")
print(Mobil.total_mobil())
print(Mobil.sewa_termurah())
```

Static Method

- **Static method** atau method statis, yang ciri khasnya adalah memiliki decorator **@staticmethod** dan **tidak memiliki parameter implicit self maupun cls**.
- Sehingga static method tidak bergantung pada object maupun class

```
class Mobil:
    jumlah_mobil = 0
    def __init__(self, nama_merek, model, tahun_produksi):
        self.nama_merek = nama_merek
        self.model = model
        self.tahun_produksi = tahun_produksi
        Mobil.jumlah_mobil += 1

    # Instance Method
    def tampilkan_info(self):
        print(f"Merek : {self.nama_merek}")
        print(f"Model : {self.perusahaan}")
        print(f"Tahun : {self.tahun_produksi}")

    # Class Method
    @classmethod
    def total_mobil(cls):
        return cls.jumlah_mobil

    # Static Method
    @staticmethod
    def validasi_nomor_polisi(nomor):
        return len(nomor) >= 5
```



```
print(f"Thank You, see you {Student.name}!")  
print(f"Don't forget your {Student.assignment}!!")  
print(f"See you in {Student.next_class}! :)")
```